

MODULE 35 · EXAM STRATEGY - ALL DOMAINS

Practice exam 2

25 further questions across all seven domains. No topic repeated from paper 1.

QUESTIONS 25 FORMAT 4 options, A–D MODULE 35 of 36

How to take this paper

- ◆ **Each question is read aloud, then the slide holds** while you decide.
- ◆ **Pause if you need longer.** The hold is a prompt, not a limit.
- ◆ **Answer every question,** even when unsure. There is no penalty for a wrong answer.
- ◆ **The worked answer follows immediately,** with a reason for each wrong option.

ON THE EXAM

The real exam is 45 questions in 90 minutes — two minutes each. Try to keep near that pace.

A data science team needs a cluster running Databricks Runtime for Machine Learning, with GPU instances, to train a model. They also want Unity Catalog governance on the data they read.

A platform engineer configures the cluster with standard access mode and the team reports that it will not start with the ML runtime.

What is required?

- ☐ A Standard access mode with the ML libraries installed manually
- ☐ B Dedicated access mode, which is required for Runtime ML, GPU workloads, the RDD API and R
- ☐ C A serverless SQL warehouse, which supports GPU workloads
- ☐ D No isolation shared access mode, which supports all runtimes

◆ PAUSE AND DECIDE

What is required?

- A

 Standard access mode with the ML libraries installed manually
The requirement is the ML runtime itself, not a set of libraries. Access mode is what blocks it.
- B

 Dedicated access mode, which is required for Runtime ML, GPU workloads, the RDD API and R
Correct. Dedicated was formerly called single user, and it is the mode those workloads require.
- C

 A serverless SQL warehouse, which supports GPU workloads
SQL warehouses run SQL. They do not run training workloads or GPU instances.
- D

 No isolation shared access mode, which supports all runtimes
It bypasses Unity Catalog entirely, so the governance requirement is lost. It is also legacy.

THE SIGNAL

Four workloads need dedicated: Runtime ML, GPU, the RDD API and R. Dedicated still supports Unity Catalog.

A COPY INTO job loads CSV files from a supplier. Some files carry an extra column that others do not. The load fails on schema mismatch between the source files themselves.

An engineer adds mergeSchema to COPY_OPTIONS. The same failure occurs.

What is wrong?

- ☐ A mergeSchema is not supported by COPY INTO at all
- ☐ B COPY INTO cannot load CSV files with differing schemas under any configuration
- ☐ C The FILES clause must be used instead of PATTERN when schemas differ
- ☐ D mergeSchema in COPY_OPTIONS evolves the target table; reconciling differing source files needs it in FORMAT_OPTIONS

◆ PAUSE AND DECIDE

What is wrong?

- A** mergeSchema is not supported by COPY INTO at all

It is supported, in both option blocks. Which block you put it in determines what it reconciles.
- B** COPY INTO cannot load CSV files with differing schemas under any configuration

It can, with FORMAT_OPTIONS mergeSchema, which is exactly the fix.
- C** The FILES clause must be used instead of PATTERN when schemas differ

FILES and PATTERN control which files are selected, not how their schemas are reconciled.
- D** mergeSchema in COPY_OPTIONS evolves the target table; reconciling differing source files needs it in FORMAT_OPTIONS

Correct. The same option name appears in both blocks and does two different jobs.

THE SIGNAL

One option name, two blocks, two jobs. FORMAT_OPTIONS reconciles source files; COPY_OPTIONS evolves the target.

A team runs two Auto Loader streams, one over an orders directory and one over a payments directory. To simplify configuration they point both at the same checkpoint location.

Both streams run without error. Weeks later, reconciliation shows that a substantial number of files from both sources were never processed.

What happened?

- ☐ A Auto Loader cannot read two directories in the same workspace concurrently
- ☐ B Each source location requires its own checkpoint; sharing one causes silent data loss
- ☐ C The schema location must also be shared for the checkpoint to work
- ☐ D The streams need `cloudFiles.includeExistingFiles` set to true

◆ PAUSE AND DECIDE

What happened?

- A** Auto Loader cannot read two directories in the same workspace concurrently
It can. Many concurrent streams are normal; each simply needs its own checkpoint.
- B** Each source location requires its own checkpoint; sharing one causes silent data loss
Correct. The checkpoint records processing progress, so two streams sharing it overwrite each other's position.
- C** The schema location must also be shared for the checkpoint to work
Sharing more state makes the problem worse. Both must be per stream.
- D** The streams need `cloudFiles.includeExistingFiles` set to true
That option controls whether pre-existing files are processed on first start. It does not fix a shared checkpoint.

THE SIGNAL

No error is raised, which is what makes it dangerous. The checkpoint is progress state, and it is per stream.

A regulated ingestion stream must never write a record whose schema it does not fully understand. If an unexpected column appears, processing must stop and stay stopped until an engineer has reviewed and approved the change.

The team is choosing an Auto Loader schema evolution mode.

Which mode fits?

- ☐ A rescue, which captures unexpected columns without failing
- ☐ B none, which ignores new columns entirely
- ☐ C failOnNewColumns, which fails and will not restart until the schema is updated or the file removed
- ☐ D addNewColumns, which fails the stream and then resumes on restart

◆ PAUSE AND DECIDE

Which mode fits?

- A** rescue, which captures unexpected columns without failing
The stream never stops, so records with unreviewed schemas are written. Wrong for this requirement.
- B** none, which ignores new columns entirely
Processing continues and the new column is silently discarded. The opposite of what is required.
- C** failOnNewColumns, which fails and will not restart until the schema is updated or the file removed
Correct. It is the only mode that stops and stays stopped, which is what the review requirement demands.
- D** addNewColumns, which fails the stream and then resumes on restart
It resumes automatically on restart, so processing continues without anybody approving the change.

THE SIGNAL

'Stop and stay stopped' is the deciding phrase. addNewColumns stops and then starts again by itself.

A team lands deeply nested JSON in a VARIANT column, which works well. They then try to improve query performance by adding the VARIANT column as a clustering key on the table. The statement is rejected.

Why, and what should they do?

- ☐ A VARIANT requires liquid clustering to be disabled on the table
- ☐ B Clustering keys must be declared at table creation and cannot be added later
- ☐ C The VARIANT column must be cast to STRING before clustering
- ☐ D VARIANT columns cannot be clustering keys, partition columns or Z-order keys; extract frequently queried fields into typed columns

◆ PAUSE AND DECIDE

Why, and what should they do?

- A

 VARIANT requires liquid clustering to be disabled on the table
Liquid clustering works on the table. The restriction is on VARIANT columns being used as keys.
- B

 Clustering keys must be declared at table creation and cannot be added later
Liquid clustering keys can be changed without rewriting the table. That is one of its main advantages.
- C

 The VARIANT column must be cast to STRING before clustering
That discards the structure VARIANT exists to preserve, and reintroduces JSON string parsing.
- D

 VARIANT columns cannot be clustering keys, partition columns or Z-order keys; extract frequently queried fields into typed columns
Correct. The documented pattern is to extract hot fields alongside the VARIANT, keeping both.

THE SIGNAL

VARIANT also cannot be used for comparisons, grouping, ordering or set operations. Extract what you query often.

A silver build explodes an `order_lines` array from a bronze orders table, producing one row per line item. Reconciliation shows the line item count is correct, but 3% of orders have disappeared entirely.

Those orders are ones placed with no line items, where the array is empty.

What should be changed?

- ☐ A Use `explode_outer`, which keeps rows whose array is empty or null
- ☐ B Filter out empty arrays before exploding so the count reconciles
- ☐ C Use a left join back to bronze after exploding
- ☐ D Use `posexplode`, which returns the position as well as the value

◆ PAUSE AND DECIDE

What should be changed?

- A

 Use `explode_outer`, which keeps rows whose array is empty or null
Correct. `explode` silently drops those rows; `explode_outer` keeps them with null in the exploded column.
- B

 Filter out empty arrays before exploding so the count reconciles
That removes the rows deliberately, which is the current behaviour. The requirement is to keep them.
- C

 Use a left join back to bronze after exploding
It would restore the rows and is far more work than changing one function name.
- D

 Use `posexplode`, which returns the position as well as the value
`posexplode` adds position information. It drops empty arrays just as `explode` does.

THE SIGNAL

Explode changes the grain and silently drops empty arrays. That is a documented behaviour, not a bug.

A team is designing bronze. To save storage they propose deduplicating, dropping malformed records and standardising timestamps as the data is written into bronze.

A colleague objects.

What is the strongest objection?

- ☐ A Deduplication is computationally expensive and would slow ingestion
- ☐ B Bronze tables cannot have transformations applied to them
- ☐ C Storage is cheap, so the saving is not worth pursuing
- ☐ D Bronze should be append-only and hold data as it arrived, so that silver can be rebuilt when the cleaning logic is found to be wrong

◆ PAUSE AND DECIDE

What is the strongest objection?

- A

 Deduplication is computationally expensive and would slow ingestion
It costs something, and cost is not the reason. The reason is losing the ability to reprocess.
- B

 Bronze tables cannot have transformations applied to them
There is no such restriction. It is a design convention, and a well-motivated one.
- C

 Storage is cheap, so the saving is not worth pursuing
True and beside the point. The objection is about reprocessing, not economics.
- D

 Bronze should be append-only and hold data as it arrived, so that silver can be rebuilt when the cleaning logic is found to be wrong
Correct. Cleaning on the way in destroys the original, and the original is what makes reprocessing possible.

THE SIGNAL

Every cleaning decision is a decision that might be wrong. Bronze is what lets you make it again.

A DataFrame with columns (txn_id, merchant_ref, terminal_ref) is written into an existing table whose columns are (txn_id, terminal_ref, merchant_ref). All three columns are strings. The write succeeds with no error. Two months later, fraud analysis is producing nonsense because merchant and terminal references are transposed.

What was used, and what should have been?

- ☐ A insertInto was used; saveAsTable matches by name and would have aligned them
- ☐ B append mode was used; overwrite would have corrected the ordering
- ☐ C saveAsTable was used; insertInto matches by name
- ☐ D The table schema must be altered to match the DataFrame's column order

◆ PAUSE AND DECIDE

What was used, and what should have been?

A insertInto was used; saveAsTable matches by name and would have aligned them

Correct. insertInto matches by position, and three string columns meant no type error surfaced.

C saveAsTable was used; insertInto matches by name

The pair stated backwards. saveAsTable matches by name; insertInto matches by position.

B append mode was used; overwrite would have corrected the ordering

Write mode governs what happens to existing rows, not how columns are matched.

D The table schema must be altered to match the DataFrame's column order

Reordering a table to suit one writer is fragile. Matching by name solves it properly.

THE SIGNAL

'All three are strings' is doing the work. A type mismatch would have failed the write immediately.

A team migrates a SQL pipeline to PySpark. The original combined four regional tables with UNION. The engineer translates it as `df_a.union(df_b).union(df_c).union(df_d)`.

The PySpark output has more rows than the SQL version. A small number of customers trade in two regions and appear in two source tables.

What is the fix?

- ☐ A Use `unionByName()`, which removes duplicate rows
- ☐ B DataFrame `union()` does not deduplicate; append `.distinct()` to match SQL UNION
- ☐ C Deduplicate each regional table before combining them
- ☐ D Use `unionAll()`, which deduplicates correctly

◆ PAUSE AND DECIDE

What is the fix?

- A** Use `unionByName()`, which removes duplicate rows
`unionByName` changes how columns are matched, from position to name. It says nothing about duplicates.
- B** `DataFrame union()` does not deduplicate; append `.distinct()` to match SQL UNION
Correct. The docs state it performs the union with no automatic deduplication.
- C** Deduplicate each regional table before combining them
The duplicates are across sources, not within them. Each table is internally correct.
- D** Use `unionAll()`, which deduplicates correctly
`unionAll` is an alias for `union`. Identical behaviour, so this changes nothing at all.

THE SIGNAL

SQL UNION deduplicates. The DataFrame method does not, and neither does `unionAll`.

A report should list every customer whose total spend across the year exceeds fifty thousand. An engineer filters where amount is greater than fifty thousand, then groups by customer, then sums.

The report returns very few customers, and their totals are far below their real annual spend.

What is wrong?

- ☐ A The comparison should be greater than or equal to
- ☐ B sum ignores nulls, so some amounts were excluded
- ☐ C The filter runs before the grouping, so only individual transactions over fifty thousand survive to be summed
- ☐ D The grouping should have been a window function

◆ PAUSE AND DECIDE

What is wrong?

- A** The comparison should be greater than or equal to
A boundary difference would change the result by a handful of customers, not produce systematically low totals.
- B** sum ignores nulls, so some amounts were excluded
True of sum, and it would not produce this pattern, nor explain the missing customers.
- C** The filter runs before the grouping, so only individual transactions over fifty thousand survive to be summed
Correct. Filtering first changes which rows are aggregated, so both the customer list and the totals are wrong.
- D** The grouping should have been a window function
A window keeps every row, which is not what a customer-level report wants. The filter's position is the problem.

THE SIGNAL

Two symptoms together: too few customers AND wrong totals. Filtering after grouping would change only the first.

A team ingests events from a Kafka topic that retains messages for three days. They want a gold object holding a running total per product, covering the last eighteen months.

An engineer proposes a materialised view defined directly over the Kafka source, refreshed every fifteen minutes.

What is wrong with the proposal?

- ☐ A Materialised views cannot compute running totals
- ☐ B Materialised views cannot be refreshed more often than hourly
- ☐ C Fifteen minutes is too slow for clickstream data; it needs continuous processing
- ☐ D A materialised view may recompute from source, and Kafka retains only three days, so older history would be lost

◆ PAUSE AND DECIDE

What is wrong with the proposal?

- A

 Materialised views cannot compute running totals
Aggregation is one of the main reasons to use one.
- B

 Materialised views cannot be refreshed more often than hourly
Invented limit. The refresh schedule is configurable.
- C

 Fifteen minutes is too slow for clickstream data; it needs continuous processing
Latency was never stated as a requirement. The problem is data loss, not freshness.
- D

 A materialised view may recompute from source, and Kafka retains only three days, so older history would be lost
Correct. Databricks documents this: do not define a materialised view against a source where records should only be processed once.

THE SIGNAL

It would appear to work for three days, then silently lose everything older on the first full recompute.

A regulated silver table must never contain a row with a null policy_number. It is currently written by one Lakeflow pipeline that has an expectation with ON VIOLATION FAIL UPDATE.

The platform team is concerned that another team may later write to this table from a notebook, bypassing the pipeline.

What should they add?

- ☐ A A NOT NULL constraint on the table, which Delta enforces against every writer
- ☐ B A scheduled query that checks for nulls and alerts the team
- ☐ C A second expectation with ON VIOLATION DROP ROW as a fallback
- ☐ D Revoke MODIFY on the table from everyone except the pipeline's service principal

◆ PAUSE AND DECIDE

What should they add?

- A

 A NOT NULL constraint on the table, which Delta enforces against every writer
Correct. A constraint is a property of the table, enforced at write time regardless of which tool performs the write.
- B

 A scheduled query that checks for nulls and alerts the team
Detection after the fact. The requirement is that the row must never exist.
- C

 A second expectation with ON VIOLATION DROP ROW as a fallback
Expectations only run inside the pipeline. Two of them protect the same single path.
- D

 Revoke MODIFY on the table from everyone except the pipeline's service principal
Reasonable defence in depth, and it does not prevent the pipeline's own identity writing a null.

THE SIGNAL

'Regardless of the writer' points at the table, not at any pipeline filling it.

A platform team reviews the previous week's job runs by filtering the Runs tab to Failed. Nothing appears and they report a clean week.

An analyst separately reports that a reconciliation table has been missing two regions since Tuesday. The loading job shows Succeeded with failures on every run since then.

What is happening?

- ☐ A The analyst is looking at a different job from the one the team reviewed
- ☐ B The job status is incorrect and should be raised with support
- ☐ C Succeeded with failures means some tasks failed but all leaf tasks succeeded; a Failed filter does not show it
- ☐ D The runs are older than the 60-day retention window and have been removed

◆ PAUSE AND DECIDE

What is happening?

- A** The analyst is looking at a different job from the one the team reviewed
The scenario states the loading job's status directly.
- B** The job status is incorrect and should be raised with support
It is exactly correct. Job outcome comes from leaf tasks, and the leaf tasks succeeded.
- C** Succeeded with failures means some tasks failed but all leaf tasks succeeded; a Failed filter does not show it
Correct. It is a distinct outcome, and filtering by Failed hides every handled failure in the estate.
- D** The runs are older than the 60-day retention window and have been removed
They are from this week, and their status is visible, so they are plainly still in history.

THE SIGNAL

The filter created the blind spot, not the platform. Job status is determined by leaf tasks.

A job's fourth task appends the day's records to a fact table. One night it fails part way through, having written roughly 70% of the batch. An engineer uses Repair run, which succeeds.

The next morning the fact table holds about 170% of the expected rows for that date.

What happened?

- ☐ A Repair re-runs each unsuccessful task from the beginning, and because the task appends, the partial output was duplicated
- ☐ B Repair re-ran every task in the job, including the successful ones
- ☐ C A retry policy fired concurrently with the repair, running the task twice
- ☐ D Delta failed to roll back the partial write, which is a known defect

◆ PAUSE AND DECIDE

What happened?

- A** Repair re-runs each unsuccessful task from the beginning, and because the task appends, the partial output was duplicated
Correct. Lakeflow Jobs does not make tasks idempotent, so a partial append is added to twice.
- B** Repair re-ran every task in the job, including the successful ones
The opposite of what repair does. Successful tasks and their dependents are deliberately not re-run.
- C** A retry policy fired concurrently with the repair, running the task twice
Invents a concurrent execution. 170% matches one partial write plus one full write.
- D** Delta failed to roll back the partial write, which is a known defect
Delta writes are atomic per write. Batches that committed before the failure are correctly committed.

THE SIGNAL

70% written, then 100% written again, gives 170%. The arithmetic identifies the cause.

A partner delivers around 300 small files in a burst lasting about five minutes, three times a day. A file arrival trigger is configured on the volume with no advanced options set.

The job runs close to 300 times per delivery, mostly against partial data, and compute cost has become the team's largest line item.

What should be configured?

- ☐ A Minimum time between triggers, set to a few minutes
- ☐ B Switch to a scheduled trigger three times a day
- ☐ C Wait after last change, set longer than the gap between files within a burst
- ☐ D Maximum concurrent runs, set to 1

◆ PAUSE AND DECIDE

What should be configured?

- A

 Minimum time between triggers, set to a few minutes
A cooldown caps how often runs start, which helps cost. Runs would still fire against partial data.
- B

 Switch to a scheduled trigger three times a day
Abandons the data-driven approach and reintroduces the margin problem, assuming punctual deliveries.
- C

 Wait after last change, set longer than the gap between files within a burst
Correct. Debouncing waits for quiet, and each new file resets the timer, so one run fires after the whole burst lands.
- D

 Maximum concurrent runs, set to 1
Already the default. It prevents overlapping runs, not repeated ones, so 300 runs would simply queue.

THE SIGNAL

Two symptoms: too many runs, and partial data. A cooldown addresses the first only; debouncing addresses both.

A Lakeflow declarative pipeline runs in continuous mode, ingesting from a message bus. A team wants a notebook task to run after the pipeline finishes processing each batch.

They configure a job with a pipeline task pointing at that pipeline, then a notebook task depending on it.

What is the problem?

- ☐ A A notebook task cannot depend on a pipeline task
- ☐ B Continuous pipelines are not supported as job tasks; the pipeline must be triggered
- ☐ C The pipeline task must be the last task in the job
- ☐ D The notebook task needs Run if set to All done

◆ PAUSE AND DECIDE

What is the problem?

- A

 A notebook task cannot depend on a pipeline task
It can. Any task may depend on any other; that is what the DAG is for.
- B

 Continuous pipelines are not supported as job tasks; the pipeline must be triggered
Correct. A continuous pipeline is already running, so there is nothing to trigger and no completion to wait on.
- C

 The pipeline task must be the last task in the job
Invented ordering rule. Task order comes entirely from declared dependencies.
- D

 The notebook task needs Run if set to All done
Run if governs behaviour once dependencies finish. A continuous pipeline never finishes.

THE SIGNAL

A continuous pipeline never completes, so there is no completion event for a downstream task to wait on.

An engineer hard-codes a storage account key into a notebook while debugging and pushes it to the team's repository. She notices an hour later.

She removes the key, commits again with the message 'remove key', pushes, and reports the issue resolved.

Is it resolved?

- ☐ A Yes; the current version of the file no longer contains the key
- ☐ B Yes, provided the repository is private
- ☐ C No; she should delete the notebook from the repository instead
- ☐ D No; the key remains in the commit history and must be rotated. Rewriting history is a separate question.

◆ PAUSE AND DECIDE

Is it resolved?

- A** Yes; the current version of the file no longer contains the key
Git stores history. The previous commit still contains the key and is readable by anyone with access.
- B** Yes, provided the repository is private
Private limits who can read it and does not un-expose it. Everyone with access, now or later, can read that commit.
- C** No; she should delete the notebook from the repository instead
Deleting a file removes it from the current version and not from history. Same mistake, more disruption.
- D** No; the key remains in the commit history and must be rotated. Rewriting history is a separate question.
Correct. The credential is compromised the moment it is pushed, and rotation is the only action that invalidates it.

THE SIGNAL

Git stores history, not just current state. Rotate first; history rewriting is legitimate and secondary.

Five engineers share a development workspace and each deploys the same bundle from their own branch to test changes. The target is configured with mode: production, chosen because it sounded more rigorous.

They keep overwriting each other's jobs, and whoever deployed most recently owns every job in the workspace.

What should they change?

- ☐ A Give each engineer their own workspace
- ☐ B Have one engineer deploy on behalf of the others
- ☐ C Add a variable for the job name and have each engineer override it
- ☐ D Set mode: development on that target, which prefixes resource names with the deploying user

◆ PAUSE AND DECIDE

What should they change?

- A

 Give each engineer their own workspace
Would work, at considerable cost and administration, when a feature exists that solves it without either.
- B

 Have one engineer deploy on behalf of the others
Solves collisions by removing the ability to test independently, which is what a dev environment is for.
- C

 Add a variable for the job name and have each engineer override it
Reimplements development mode by hand, and depends on every engineer setting it correctly every time.
- D

 Set mode: development on that target, which prefixes resource names with the deploying user
Correct. Development mode prefixes resources not deployed as files or notebooks, so five engineers get five sets of jobs.

THE SIGNAL

Modes are not a quality dial. They select behaviours suited to a purpose, and prefixing is development mode's.

A bundle deploys correctly from an engineer's laptop with databricks bundle deploy -t dev. The same command in a CI job fails immediately, reporting that the CLI is not configured. The CI secret store holds a valid service principal client id and client secret.

What is missing?

- ☐ A The CI runner needs the Databricks CLI reinstalled
- ☐ B The credentials are in the secret store but are not exposed as environment variables for the step
- ☐ C The bundle needs a profile name added to the target definition
- ☐ D The service principal has not been granted CAN MANAGE on the bundle

◆ PAUSE AND DECIDE

What is missing?

- A** The CI runner needs the Databricks CLI reinstalled
The CLI ran and produced a configuration error, so it is installed and executing.
- B** The credentials are in the secret store but are not exposed as environment variables for the step
Correct. A fresh CI container has no .databrickscfg, and environment variables take precedence for OAuth M2M.
- C** The bundle needs a profile name added to the target definition
A profile name refers to an entry in .databrickscfg, and the runner has no such file.
- D** The service principal has not been granted CAN MANAGE on the bundle
A permissions problem produces an authorisation error, not 'not configured'.

THE SIGNAL

'Works on my laptop' is the clue. The laptop has a .databrickscfg; a container created 30 seconds ago has nothing.

A stage takes 38 minutes across 200 tasks. Summary Metrics show a median task duration of 10.2 minutes, a 75th percentile of 10.9 minutes, and a maximum of 11.4 minutes.

The cluster has 24 cores. No spill is reported.

What is the bottleneck?

- ☐ A Too many shuffle partitions, creating scheduling overhead
- ☐ B Memory pressure, indicated by the long task durations
- ☐ C Not skew; the distribution is even and 200 tasks on 24 cores means the cluster is under-resourced for the work
- ☐ D Data skew, since the maximum exceeds the median

◆ PAUSE AND DECIDE

What is the bottleneck?

- A** Too many shuffle partitions, creating scheduling overhead
Excessive partitions produce many tiny tasks. These take over ten minutes each, the opposite symptom.
- B** Memory pressure, indicated by the long task durations
Memory pressure shows as spill or GC time, and the scenario states there is no spill.
- C** Not skew; the distribution is even and 200 tasks on 24 cores means the cluster is under-resourced for the work
Correct. Every task is slow, evenly, which is the signature of capacity rather than skew.
- D** Data skew, since the maximum exceeds the median
11.4 against 10.9 minutes is about 5% above the 75th percentile, far below the 50% threshold.

THE SIGNAL

A few slow tasks is skew. All tasks slow is not, and the fixes point in opposite directions.

A stage reports shuffle spill of 52 GB in memory and 8 GB on disk. Task durations are fairly even, with the maximum about 15% above the 75th percentile.

An engineer proposes moving to a node type with four times the memory.

What should be tried first?

- ☐ A The four-times-memory node type, since spill means insufficient memory
- ☐ B Increase `spark.sql.shuffle.partitions` so each partition is smaller and fits the memory available
- ☐ C Nothing; 52 GB against 8 GB shows the metrics are inconsistent
- ☐ D Enable AQE to split the skewed partitions

◆ PAUSE AND DECIDE

What should be tried first?

- A

 The four-times-memory node type, since spill means insufficient memory
It would help, at four times the cost, permanently. Spill is as often partitions being too large.
- B

 Increase `spark.sql.shuffle.partitions` so each partition is smaller and fits the memory available
Correct. Smaller partitions remove the spill without changing the cluster, and it is the cheaper fix.
- C

 Nothing; 52 GB against 8 GB shows the metrics are inconsistent
They are consistent. Memory spill measures deserialised data; disk spill measures the serialised form.
- D

 Enable AQE to split the skewed partitions
There is no skew. 15% above the 75th percentile is under the 50% threshold.

THE SIGNAL

Memory spill much larger than disk spill is normal. Deserialised data is bigger than serialised.

An engineer adds a cluster-scoped init script that installs a package from an internal repository. On his two-node development cluster it works every time.

On the sixty-node production cluster the launch fails with `INIT_SCRIPT_FAILURE`, but not always — roughly one launch in four succeeds.

What is the most likely explanation?

- ☐ A Init scripts run on every node, so a transient failure reaching the repository on any one of sixty nodes terminates the cluster
- ☐ B The production cluster has an incompatible runtime version
- ☐ C The script should be converted to a global init script
- ☐ D Init scripts are not supported on clusters above a certain size

◆ PAUSE AND DECIDE

What is the most likely explanation?

- A** Init scripts run on every node, so a transient failure reaching the repository on any one of sixty nodes terminates the cluster
Correct. A per-node success rate that is fine on two nodes becomes unreliable on sixty, because all sixty must succeed.
- B** The production cluster has an incompatible runtime version
A version incompatibility fails consistently, not one launch in four. Intermittency points at something probabilistic.
- C** The script should be converted to a global init script
Changes who manages it and applies it workspace-wide. It does not change the per-node execution that is failing.
- D** Init scripts are not supported on clusters above a certain size
Invented limit. Init scripts run on clusters of any size.

THE SIGNAL

Two nodes and sixty nodes are not the same test. Thirty times the nodes is thirty times the chances of one failing.

A platform team owns every catalog and has become a bottleneck for access requests. They want team leads to grant access to their own team's tables, without being able to drop those tables or transfer ownership.

An engineer proposes granting ALL PRIVILEGES on the relevant schemas.

What should they grant instead?

- ☐ A MANAGE on the relevant schemas
- ☐ B Metastore admin, so they can administer their own data
- ☐ C ALL PRIVILEGES as proposed
- ☐ D Ownership of each schema, transferred to the relevant team lead

◆ PAUSE AND DECIDE

What should they grant instead?

- A

MANAGE on the relevant schemas
Correct. MANAGE confers exactly the ability to grant and revoke privileges, without ownership's other capabilities.
- B

Metastore admin, so they can administer their own data
Grants administrative reach over the entire metastore to solve a per-schema problem.
- C

ALL PRIVILEGES as proposed
ALL PRIVILEGES deliberately excludes MANAGE, so it would not let them grant to others – the one thing they need.
- D

Ownership of each schema, transferred to the relevant team lead
Owners can also drop the object and transfer ownership onward, which the requirement excludes.

THE SIGNAL

ALL PRIVILEGES sounds like more than **MANAGE** and specifically excludes it, to prevent privilege escalation.

A sales table is queried by fourteen regional teams. Each team must see only rows for its own region, and a global team must see every row. Every team must be able to query the same table by name.

Row counts per team will naturally differ, and that is expected.

What should be applied?

- ☐ A A row filter on the table, using a function that returns a boolean based on group membership
- ☐ B Revoke SELECT and grant it per region on separate copies of the table
- ☐ C Fourteen dynamic views, one per region
- ☐ D A column mask on the region column

◆ PAUSE AND DECIDE

What should be applied?

- A

 A row filter on the table, using a function that returns a boolean based on group membership
Correct. A row filter returns a boolean and removes rows the user should not see, evaluated per row at query time.
- B

 Revoke SELECT and grant it per region on separate copies of the table
Fourteen copies to keep in sync, which is what query-time filtering exists to avoid.
- C

 Fourteen dynamic views, one per region
Would work, and teams must then query different objects, which the requirement forbids.
- D

 A column mask on the region column
A mask transforms a value and keeps every row. It would hide the region name while still returning all rows.

THE SIGNAL

Hide rows: row filter, returns boolean. Hide a value but keep the row: column mask, returns the column's type.

A team is migrating from table-level column masks to ABAC policies. On one table they add an ABAC column mask policy while the existing ALTER TABLE mask is still in place. The two use different functions, though both redact to the same fixed string.

Queries against that table now fail with an error rather than returning masked data.

What is happening?

- ☐ A ABAC requires the table-level mask to be dropped before the policy is created
- ☐ B The two masks are applied in sequence, double-masking the value
- ☐ C When both mechanisms apply different functions, access is blocked — even if the output would be identical
- ☐ D ABAC policies cannot be applied to tables that already have masks, so the policy was never created

◆ PAUSE AND DECIDE

What is happening?

- A** ABAC requires the table-level mask to be dropped before the policy is created
Creation succeeds either way. The conflict surfaces at query time, not at creation.
- C** When both mechanisms apply different functions, access is blocked — even if the output would be identical
Correct. The same function on both sides is allowed; different functions are refused, and the system does not compare outputs.
- B** The two masks are applied in sequence, double-masking the value
The system does not chain them. It refuses rather than composing two protections it cannot reconcile.
- D** ABAC policies cannot be applied to tables that already have masks, so the policy was never created
Both can coexist. The same function on both sides is explicitly allowed, so creation is not what failed.

THE SIGNAL

'Both redact to the same fixed string' is placed to tempt you. The system compares functions, not outputs.

What this paper covered

DOMAIN	QUESTIONS
Domain 1 – Platform	1
Domain 2 – Ingest	6
Domain 3 – Transform	5
Domain 4 – Jobs	4
Domain 5 – CI/CD	3
Domain 6 – Troubleshoot	3
Domain 7 – Govern	3
Total	25

ON THE EXAM

Weighted across all three papers rather than within each one, so a single paper will not match the exam's weights exactly.